

TP2 — Concurrency e IA: evidencia de código y salidas reales

Captura por sección con Win+Shift+S · Código de restricciones.sql y test_restricciones.sql + salidas reales del motor

TP2 · Parte 1 · restricciones.sql — Regla 1: baja lógica (no DELETE físico)

Function + triggers trg_categoria_no_delete y trg_producto_no_delete (líneas 15-31)

```
-- =====
-- FOOD STORE - restricciones.sql
-- TP2 - Parte 1: Integridad versionada
-- =====
-- Reglas de negocio que hoy dependen de la aplicacion y no
-- estan garantizadas por el motor. Se agregan como triggers.
-- =====

-- =====
-- REGLA 1: Baja logica - impedir DELETE directo
-- Solo se permite desactivar con UPDATE ... SET activo = FALSE,
-- nunca borrar fisicamente un registro de categoria o producto.
-- =====

CREATE OR REPLACE FUNCTION fn_impedir_delete_logico()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'No esta permitido eliminar registros fisicamente. Use UPDATE ... SET activo =
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_categoria_no_delete
    BEFORE DELETE ON categoria
    FOR EACH ROW
    EXECUTE FUNCTION fn_impedir_delete_logico();

CREATE TRIGGER trg_producto_no_delete
    BEFORE DELETE ON producto
    FOR EACH ROW
    EXECUTE FUNCTION fn_impedir_delete_logico();
```

TP2 · Parte 1 · restricciones.sql — Regla 2: stock no negativo al cargar detalle

Function fn_descontar_stock + trigger trg_detalle_descontar_stock (líneas 38-59)

```
-- =====  
-- REGLA 2: Stock al crear detalle - decrementar stock y  
-- verificar que no quede negativo  
-- =====  
  
CREATE OR REPLACE FUNCTION fn_descontar_stock()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF (SELECT stock FROM producto WHERE id = NEW.producto_id) < NEW.cantidad THEN  
        RAISE EXCEPTION 'Stock insuficiente para el producto id=%. Stock actual: %, cantidad pedido: %,  
            NEW.producto_id,  
            (SELECT stock FROM producto WHERE id = NEW.producto_id),  
            NEW.cantidad;  
    END IF;  
  
    UPDATE producto  
    SET stock = stock - NEW.cantidad  
    WHERE id = NEW.producto_id;  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_detalle_descontar_stock  
    BEFORE INSERT ON detalle_pedido  
    FOR EACH ROW  
    EXECUTE FUNCTION fn_descontar_stock();
```

TP2 · Parte 1 · restricciones.sql — Regla 3: subtotal = cantidad × precio_unitario

Function fn_verificar_subtotal + trigger trg_detalle_verificar_subtotal (líneas 65-83)

```
-- =====  
-- REGLA 3: Subtotal consistente - cantidad x precio_unitario  
-- =====  
  
CREATE OR REPLACE FUNCTION fn_verificar_subtotal()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.subtotal != (NEW.cantidad * NEW.precio_unitario) THEN  
        RAISE EXCEPTION 'El subtotal (%) no coincide con cantidad (%) x precio_unitario (%). Resultado:  
            NEW.subtotal,  
            NEW.cantidad,  
            NEW.precio_unitario,  
            (NEW.cantidad * NEW.precio_unitario);  
    END IF;  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_detalle_verificar_subtotal  
BEFORE INSERT OR UPDATE ON detalle_pedido  
FOR EACH ROW  
EXECUTE FUNCTION fn_verificar_subtotal();
```

TP2 · Parte 1 · test_restricciones.sql — tests dentro de BEGIN...ROLLBACK con SAVEPOINT (protocolo transacción)

Tests validos e invalidos de las 3 reglas, todo revertido

```
-- =====
-- FOOD STORE - test_restricciones.sql
-- Prueba las restricciones dentro de una transaccion
-- Usa SAVEPOINT para que un error no aborte el resto de los tests
-- =====
-- Ejecutar desde la terminal:
-- psql -d copia_trabajo -f test_restricciones.sql
-- Resultado: los 3 primeros errores son ESPERADOS (reglas que
-- rechazan datos invalidos). Al final va ROLLBACK.
-- =====

BEGIN;

-- =====
-- SETUP: datos de prueba
-- =====

INSERT INTO categoria (nombre) VALUES ('Pizzas');
INSERT INTO categoria (nombre) VALUES ('Bebidas');

INSERT INTO producto (nombre, precio, stock, activo, categoria_id)
VALUES ('Muzzarella', 800.00, 10, TRUE,
        (SELECT id FROM categoria WHERE nombre = 'Pizzas'));

INSERT INTO producto (nombre, precio, stock, activo, categoria_id)
VALUES ('Coca-Cola', 500.00, 20, TRUE,
        (SELECT id FROM categoria WHERE nombre = 'Bebidas'));

INSERT INTO cliente (nombre, apellido, email)
VALUES ('Juan', 'Perez', 'juan@test.com');

INSERT INTO pedido (forma_pago, cliente_id)
VALUES ('EFECTIVO',
        (SELECT id FROM cliente WHERE email = 'juan@test.com'));

-- =====
-- TEST 1: Baja logica - DELETE directo (debe FALLAR)
-- =====

SAVEPOINT test1;
DELETE FROM categoria WHERE nombre = 'Pizzas';
-- ERROR esperado: "No esta permitido eliminar registros fisicamente..."
ROLLBACK TO test1;

-- =====
```

```
-- TEST 2: Baja logica - UPDATE Logico (debe funcionar)
-- =====

UPDATE categoria SET activo = FALSE WHERE nombre = 'Bebidas';
-- OK, 1 fila afectada

-- =====
-- TEST 3: Stock - INSERT valido (debe funcionar y bajar stock)
-- =====

INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario, subtotal)
VALUES (
    (SELECT id FROM pedido ORDER BY id DESC LIMIT 1),
    (SELECT id FROM producto WHERE nombre = 'Muzzarella'),
    3, 800.00, 2400.00
);
-- OK

SELECT nombre, stock FROM producto WHERE nombre = 'Muzzarella';
-- Stock esperado: 7

-- =====
-- TEST 4: Stock - INSERT con stock insuficiente (debe FALLAR)
-- =====

SAVEPOINT test4;
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario, subtotal)
VALUES (
    (SELECT id FROM pedido ORDER BY id DESC LIMIT 1),
    (SELECT id FROM producto WHERE nombre = 'Coca-Cola'),
    100, 500.00, 50000.00
);
-- ERROR esperado: "Stock insuficiente..."
ROLLBACK TO test4;

-- =====
-- TEST 5: Subtotal - subtotal incorrecto (debe FALLAR)
-- =====

SAVEPOINT test5;
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario, subtotal)
VALUES (
    (SELECT id FROM pedido ORDER BY id DESC LIMIT 1),
    (SELECT id FROM producto WHERE nombre = 'Coca-Cola'),
    2, 500.00, 9999.00
);
-- ERROR esperado: "EL subtotal no coincide..."
ROLLBACK TO test5;

-- =====
```

```
-- TEST 6: Subtotal - subtotal correcto (debe funcionar)
-- =====

INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario, subtotal)
VALUES (
    (SELECT id FROM pedido ORDER BY id DESC LIMIT 1),
    (SELECT id FROM producto WHERE nombre = 'Coca-Cola'),
    2, 500.00, 1000.00
);
-- OK

-- =====
-- RESUMEN
-- =====

SELECT nombre, activo FROM categoria;
SELECT nombre, stock, activo FROM producto;

ROLLBACK;
```

TP2 · Parte 2 · informe_concurrencia.md — Escenario 1 (lectura no repetible)

Salidas reales: READ COMMITTED 10→50, REPEATABLE READ 10→10

Informe de Concurrencia – Food Store

Escenario 1: Lectura no repetible

Cómo se reprodujo

****Sesión A:****

```sql

BEGIN;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT stock FROM producto WHERE nombre = 'Producto Test';

```

****Sesión B:****

```sql

BEGIN;

UPDATE producto SET stock = 50 WHERE nombre = 'Producto Test';

COMMIT;

```

****Sesión A (continúa, unos segundos después):****

```sql

SELECT stock FROM producto WHERE nombre = 'Producto Test';

ROLLBACK;

```

Qué se observó

Salida real (READ COMMITTED):

Momento	Sesión A (SELECT)	Sesión B (UPDATE)
Primera lectura	stock = 10	–
–	–	UPDATE stock = 50; COMMIT
Segunda lectura	stock = 50	–

El mismo SELECT dentro de la misma transacción devolvió valores distintos: 10 y luego 50.

Explicación de la IA

Herramienta: ****OpenCode****

> "Con READ COMMITTED, cada lectura ve el último COMMIT en el momento de ejecutarse. Si otra sesión

Verificación en el motor

Salida real (REPEATABLE READ):

```
```sql
-- Sesión A con REPEATABLE READ
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT stock FROM producto WHERE nombre = 'Producto Test';
-- lectura 1 = 10

-- Sesión B
BEGIN;
UPDATE producto SET stock = 50 WHERE nombre = 'Producto Test';
COMMIT;

-- Sesión A (continúa)
SELECT stock FROM producto WHERE nombre = 'Producto Test';
-- lectura 2 = 10 (no cambió)

ROLLBACK;
```

### Conclusión

La explicación de la IA se confirmó. Con REPEATABLE READ la segunda lectura devolvió el mismo valor.

---

## Escenario 2: Lectura fantasma

### Cómo se reprodujo

**Sesión A:**
```sql
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
```

**Sesión B:**
```sql
BEGIN;
INSERT INTO categoria (nombre) VALUES ('Nueva');
INSERT INTO producto (nombre, precio, stock, activo, categoria_id)
VALUES ('Test Fantasma', 100.00, 5, TRUE, (SELECT id FROM categoria WHERE nombre = 'Nueva'));
COMMIT;
```

**Sesión A (continúa):**
```sql
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
```
```

```
COMMIT;
...
```

Qué se observó

Salida real (READ COMMITTED):

| Momento | Sesión A (COUNT) | Sesión B (INSERT) |
|-----------------|-------------------------|-------------------|
| Primera lectura | COUNT = 1 | - |
| | INSERT producto; COMMIT | |
| Segunda lectura | COUNT = 2 | - |

Apareció una fila nueva (fantasma) que cumplía la condición del WHERE.

Explicación de la IA

Herramienta: ****OpenCode****

> "Con READ COMMITTED, cada COUNT ve las filas commiteadas al momento de ejecutarse. Si otra sesión

Verificación en el motor

Salida real (SERIALIZABLE):

```
```sql
-- Sesión A con SERIALIZABLE
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
-- primer COUNT = 2

-- Sesión B
BEGIN;
INSERT INTO categoria (nombre) VALUES ('FantasmaNueva2');
INSERT INTO producto (nombre, precio, stock, activo, categoria_id)
VALUES ('FantasmaY', 100.00, 5, TRUE, (SELECT id FROM categoria WHERE nombre = 'FantasmaNueva2'));
COMMIT;

-- Sesión A (continúa)
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
-- segundo COUNT = 2 (no cambió, el fantasma no aparece)

ROLLBACK;
```
```

Conclusión

La explicación de la IA se confirmó. Con SERIALIZABLE el fantasma no aparece: el segundo COUNT dev

Escenario 3: Espera por bloqueo

Cómo se reprodujo

Sesión A:

```

```sql
BEGIN;
SELECT nombre FROM producto WHERE nombre = 'Producto Lock' FOR UPDATE;
-- (no hace COMMIT todavía; retiene el lock 5 segundos)
```

```

Sesión B:

```

```sql
BEGIN;
SELECT nombre FROM producto WHERE nombre = 'Producto Lock' FOR UPDATE;
-- SE BLOQUEA: queda esperando a que la sesión A haga COMMIT o ROLLBACK
```

```

Sesión A (tras 5 segundos):

```

```sql
ROLLBACK;
-- libera el lock, la sesión B se desbloquea
```

```

Qué se observó

Salida real:

| Momento | Sesión A | Sesión B |
|---------|--|-------------------------------------|
| T=0 | `SELECT ... FOR UPDATE` → OK, toma el lock | — |
| T=2s | (retiene el lock) | `SELECT ... FOR UPDATE` → BLOQUEADA |
| T=5s | `ROLLBACK` → libera el lock | Se desbloquea y devuelve la fila |

Medición real: la sesión B tardó **~3.4 segundos** en completar el `FOR UPDATE`, es decir, quedó e

Explicación de la IA

Herramienta: ****OpenCode****

> "FOR UPDATE toma un lock exclusivo sobre las filas que matchean el WHERE. Si otra sesión intenta

Verificación en el motor

Se ejecutó en el motor real y se confirmó que:

1. La sesión B quedó bloqueada (~3.4 segundos de espera real medidos con Stopwatch).
2. Al hacer ROLLBACK en A, B se desbloqueó y recibió la fila.

Comandos que dieron las salidas reales:

```
```sql
-- SESION A (job en segundo plano)
BEGIN;
SELECT ... FROM producto WHERE nombre = 'Producto Lock' FOR UPDATE;
SELECT pg_sleep(5);
ROLLBACK;

-- SESION B (ejecutada al mismo tiempo)
SELECT ... FROM producto WHERE nombre = 'Producto Lock' FOR UPDATE;
-- -> tarda 3.4 s (bloqueada) y recién entonces devuelve la fila
```
```

3. El nivel de aislamiento no afecta este comportamiento: el lock exclusivo por `FOR UPDATE` ocurre

Conclusión

La explicación de la IA se confirmó. El bloqueo por `FOR UPDATE` es el mecanismo esperado: la sesi

TP2 · Parte 2 · informe_concurrencia.md — Escenario 2 (lectura fantasma) y Escenario 3 (espera por lock)

Salidas reales: COUNT 1→2 (RC) y 2→2 (SERIALIZABLE); espera de bloqueo ~3.4 s

Informe de Concurrencia – Food Store

Escenario 1: Lectura no repetible

Cómo se reprodujo

****Sesión A:****

```
```sql
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT stock FROM producto WHERE nombre = 'Producto Test';
```
```

****Sesión B:****

```
```sql
BEGIN;
UPDATE producto SET stock = 50 WHERE nombre = 'Producto Test';
COMMIT;
```
```

****Sesión A (continúa, unos segundos después):****

```
```sql
SELECT stock FROM producto WHERE nombre = 'Producto Test';
ROLLBACK;
```
```

Qué se observó

Salida real (READ COMMITTED):

| Momento | Sesión A (SELECT) | Sesión B (UPDATE) |
|-----------------|-------------------|-------------------|
| Primera lectura | stock = 10 | – |
| Segunda lectura | stock = 50 | – |

El mismo SELECT dentro de la misma transacción devolvió valores distintos: 10 y luego 50.

Explicación de la IA

Herramienta: ****OpenCode****

> "Con READ COMMITTED, cada lectura ve el último COMMIT en el momento de ejecutarse. Si otra sesión

Verificación en el motor

Salida real (REPEATABLE READ):

```
```sql
-- Sesión A con REPEATABLE READ
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT stock FROM producto WHERE nombre = 'Producto Test';
-- lectura 1 = 10

-- Sesión B
BEGIN;
UPDATE producto SET stock = 50 WHERE nombre = 'Producto Test';
COMMIT;

-- Sesión A (continúa)
SELECT stock FROM producto WHERE nombre = 'Producto Test';
-- lectura 2 = 10 (no cambió)

ROLLBACK;
```

### Conclusión

La explicación de la IA se confirmó. Con REPEATABLE READ la segunda lectura devolvió el mismo valor.

---

## Escenario 2: Lectura fantasma

### Cómo se reprodujo

**Sesión A:**
```sql
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
```

**Sesión B:**
```sql
BEGIN;
INSERT INTO categoria (nombre) VALUES ('Nueva');
INSERT INTO producto (nombre, precio, stock, activo, categoria_id)
VALUES ('Test Fantasma', 100.00, 5, TRUE, (SELECT id FROM categoria WHERE nombre = 'Nueva'));
COMMIT;
```

**Sesión A (continúa):**
```sql
-- Sesión A (continúa)
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
-- resultado: 1 (incluyendo el producto insertado en Sesión B)
```
```

```
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
COMMIT;
...
```

Qué se observó

Salida real (READ COMMITTED):

| Momento | Sesión A (COUNT) | Sesión B (INSERT) |
|-----------------|------------------|-------------------------|
| Primera lectura | COUNT = 1 | - |
| | | INSERT producto; COMMIT |
| Segunda lectura | COUNT = 2 | - |

Apareció una fila nueva (fantasma) que cumplía la condición del WHERE.

Explicación de la IA

Herramienta: **OpenCode**

> "Con READ COMMITTED, cada COUNT ve las filas commiteadas al momento de ejecutarse. Si otra sesión

Verificación en el motor

Salida real (SERIALIZABLE):

```
```sql
-- Sesión A con SERIALIZABLE
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
-- primer COUNT = 2

-- Sesión B
BEGIN;
INSERT INTO categoria (nombre) VALUES ('FantasmaNueva2');
INSERT INTO producto (nombre, precio, stock, activo, categoria_id)
VALUES ('FantasmaY', 100.00, 5, TRUE, (SELECT id FROM categoria WHERE nombre = 'FantasmaNueva2'));
COMMIT;

-- Sesión A (continúa)
SELECT COUNT(*) FROM producto WHERE activo = TRUE;
-- segundo COUNT = 2 (no cambió, el fantasma no aparece)

ROLLBACK;
...
```
```

Conclusión

La explicación de la IA se confirmó. Con SERIALIZABLE el fantasma no aparece: el segundo COUNT dev

```
---
```

Escenario 3: Espera por bloqueo

Cómo se reprodujo

Sesión A:

```
```sql
BEGIN;
SELECT nombre FROM producto WHERE nombre = 'Producto Lock' FOR UPDATE;
-- (no hace COMMIT todavía; retiene el lock 5 segundos)
```
```

Sesión B:

```
```sql
BEGIN;
SELECT nombre FROM producto WHERE nombre = 'Producto Lock' FOR UPDATE;
-- SE BLOQUEA: queda esperando a que la sesión A haga COMMIT o ROLLBACK
```
```

Sesión A (tras 5 segundos):

```
```sql
ROLLBACK;
-- libera el lock, la sesión B se desbloquea
```
```

Qué se observó

Salida real:

| Momento | Sesión A | Sesión B |
|---------|--|-------------------------------------|
| T=0 | `SELECT ... FOR UPDATE` → OK, toma el lock | — |
| T=2s | (retiene el lock) | `SELECT ... FOR UPDATE` → BLOQUEADA |
| T=5s | `ROLLBACK` → libera el lock | Se desbloquea y devuelve la fila |

Medición real: la sesión B tardó **~3.4 segundos** en completar el `FOR UPDATE`, es decir, quedó e

Explicación de la IA

Herramienta: ****OpenCode****

> "FOR UPDATE toma un lock exclusivo sobre las filas que matchean el WHERE. Si otra sesión intenta

Verificación en el motor

Se ejecutó en el motor real y se confirmó que:

1. La sesión B quedó bloqueada (~3.4 segundos de espera real medidos con Stopwatch).
2. Al hacer ROLLBACK en A, B se desbloqueó y recibió la fila.

Comandos que dieron las salidas reales:

```
```sql
-- SESSION A (job en segundo plano)
BEGIN;
SELECT ... FROM producto WHERE nombre = 'Producto Lock' FOR UPDATE;
SELECT pg_sleep(5);
ROLLBACK;

-- SESSION B (ejecutada al mismo tiempo)
SELECT ... FROM producto WHERE nombre = 'Producto Lock' FOR UPDATE;
-- -> tarda 3.4 s (bloqueada) y recién entonces devuelve la fila
```
```

3. El nivel de aislamiento no afecta este comportamiento: el lock exclusivo por `FOR UPDATE` ocurre

Conclusión

La explicación de la IA se confirmó. El bloqueo por `FOR UPDATE` es el mecanismo esperado: la sesi